

- Description

commons-math-4.0-beta1-src\commons-math-
legacy\src\main\java\org\apache\commons\math4\legacy\analysis\integration\
RombergIntegrator.java

- Specification

module RombergIntegrator

type

```
relativeAccuracy: real          // relAcc
absoluteAccuracy: real         // absAcc
minimallterationCount: int      // minlter
maximallterationCount: int      // maxlter
min: real
max: real
maxEval: int
evaluations_used: int
iterations_used: int
converged_by_value_accuracy: bool
converged_by_interval_shrinkage: bool
not_converged_by_accuracy: bool
ex: set of {
    None,
    NullPointerException,
    NotStrictlyPositiveException,
    NumberIsTooSmallException,
    NumberIsTooLargeException,
    NotANumberException,
    NotFiniteNumberException,
    TooManyEvaluationsException,
    MaxCountExceededException
}
```

process integrate(f, min, max, maxEval,
relativeAccuracy, absoluteAccuracy,
minimallterationCount, maximallterationCount: real)
result: real, ex: set of { ... }

Functional Scenario Form

```
f = 0                                && ex = NullPointerException ||
maxEval <= 0                        && ex =
NotStrictlyPositiveException ||
relativeAccuracy <= 0    && ex = NotStrictlyPositiveException ||
minimallterationCount < MIN_ROMBERG_ITER    && ex =
NumberIsTooSmallException ||
```

```

    maximalIterationCount > MAX_ROMBERG_ITER                && ex =
NumberIsTooLargeException ||
    minimalIterationCount >= maximalIterationCount          && ex =
NumberIsTooSmallException ||
    isNaN=1                && ex = NotANumberException ||
    isInfinite=1           && ex = NotFiniteNumberException ||
    min >= max                && ex =
NumberIsTooLargeException ||
    evaluations_used > maxEval                && ex = TooManyEvaluationsException ||
        iterations_used >= maximalIterationCount                && ex =
MaxCountExceededException ||
    evaluations_used                <=                maxEval
    && ex = None
end_process
end_module

```

- Java Code

```

/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.integration;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveExc
eption;

```

```

import org.apache.commons.math4.legacy.exception.NumberIsTooSmallExcept
ion;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.legacy.exception.NotANumberException;
import org.apache.commons.math4.legacy.exception.NotFiniteNumberExcepti
on;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsExce
ption;
import org.apache.commons.math4.legacy.exception.MaxCountExceededExcept
ion;
import org.apache.commons.math4.core.jdkmath.JdkMath;
/**
 * Implements the <a href="http://mathworld.wolfram.com/RombergIntegrat
ion.html">
 * Romberg Algorithm</a> for integration of real univariate functions.
For
 * reference, see <b>Introduction to Numerical Analysis</b>, ISBN 03879
5452X,
 * chapter 3.
 * <p>
 * Romberg integration employs k successive refinements of the trapezoi
d
 * rule to remove error terms less than order  $O(N^{(-2k)})$ . Simpson's rule
 * is a special case of  $k = 2$ .</p>
 *
 * @since 1.2
 */
public class RombergIntegrator extends BaseAbstractUnivariateIntegrator
{
    /** Maximal number of iterations for Romberg. */
    public static final int ROMBERG_MAX_ITERATIONS_COUNT = 32;
    /**
     * Build a Romberg integrator with given accuracies and iterations
counts.
     * @param relativeAccuracy relative accuracy of the result
     * @param absoluteAccuracy absolute accuracy of the result
     * @param minimalIterationCount minimum number of iterations
     * @param maximalIterationCount maximum number of iterations
     * (must be less than or equal to {@link #ROMBERG_MAX_ITERATIONS_CO
UNT})
     * @exception org.apache.commons.math4.legacy.exception.NotStrictly
PositiveException if minimal number of iterations

```

```

    * is not strictly positive
    * @exception org.apache.commons.math4.legacy.exception.NumberIsToo
SmallException if maximal number of iterations
    * is lesser than or equal to the minimal number of iterations
    * @exception NumberIsTooLargeException if maximal number of iterat
ions
    * is greater than {@link #ROMBERG\_MAX\_ITERATIONS\_COUNT}
    */
    public RombergIntegrator(final double relativeAccuracy,
                             final double absoluteAccuracy,
                             final int minimalIterationCount,
                             final int maximalIterationCount) {
        super(relativeAccuracy, absoluteAccuracy, minimalIterationCount
, maximalIterationCount);
        if (maximalIterationCount > ROMBERG_MAX_ITERATIONS_COUNT) {
            throw new NumberIsTooLargeException(maximalIterationCount,
ROMBERG_MAX_ITERATIONS_
COUNT, false);
        }
    }
    /**
    * Build a Romberg integrator with given iteration counts.
    * @param minimalIterationCount minimum number of iterations
    * @param maximalIterationCount maximum number of iterations
    * (must be less than or equal to {@link #ROMBERG\_MAX\_ITERATIONS\_CO
UNT})
    * @exception org.apache.commons.math4.legacy.exception.NotStrictly
PositiveException if minimal number of iterations
    * is not strictly positive
    * @exception org.apache.commons.math4.legacy.exception.NumberIsToo
SmallException if maximal number of iterations
    * is lesser than or equal to the minimal number of iterations
    * @exception NumberIsTooLargeException if maximal number of iterat
ions
    * is greater than {@link #ROMBERG\_MAX\_ITERATIONS\_COUNT}
    */
    public RombergIntegrator(final int minimalIterationCount,
                             final int maximalIterationCount) {
        super(minimalIterationCount, maximalIterationCount);
        if (maximalIterationCount > ROMBERG_MAX_ITERATIONS_COUNT) {
            throw new NumberIsTooLargeException(maximalIterationCount,
ROMBERG_MAX_ITERATIONS_
COUNT, false);
        }
    }

```

```

    }
    /**
     * Construct a Romberg integrator with default settings
     * (max iteration count set to {@link #ROMBERG_MAX_ITERATIONS_COUNT
    }).
    */
    public RombergIntegrator() {
        super(DEFAULT_MIN_ITERATIONS_COUNT, ROMBERG_MAX_ITERATIONS_COUNT);
    }
    /** {@inheritDoc} */
    @Override
    protected double doIntegrate() {
        final int m = iterations.getMaximalCount() + 1;
        double[] previousRow = new double[m];
        double[] currentRow = new double[m];
        TrapezoidIntegrator qtrap = new TrapezoidIntegrator();
        currentRow[0] = qtrap.stage(this, 0);
        iterations.increment();
        double olds = currentRow[0];
        while (true) {
            final int i = iterations.getCount();
            // switch rows
            final double[] tmpRow = previousRow;
            previousRow = currentRow;
            currentRow = tmpRow;
            currentRow[0] = qtrap.stage(this, i);
            iterations.increment();
            for (int j = 1; j <= i; j++) {
                // Richardson extrapolation coefficient
                final double r = (1L << (2 * j)) - 1;
                final double tIJm1 = currentRow[j - 1];
                currentRow[j] = tIJm1 + (tIJm1 - previousRow[j - 1]) /
r;
            }
            final double s = currentRow[i];
            if (i >= getMinimalIterationCount()) {
                final double delta = JdkMath.abs(s - olds);
                final double rLimit = getRelativeAccuracy() * (JdkMath.
abs(olds) + JdkMath.abs(s)) * 0.5;
                if (delta <= rLimit || delta <= getAbsoluteAccuracy())
{
                    return s;
                }
            }
        }
    }

```

```

        }
        olds = s;
    }
}

class RIVT {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n==== " + name + " =====");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    // ----- sample functions -----
    static final UnivariateFunction F_ZERO = x -> 0.0; // integral = 0
    static final UnivariateFunction F_CONST = x -> 5.0; // ∫ 5 dx = 5(b-a)
    static final UnivariateFunction F_LIN = x -> x - 2.0; // ∫_0^4 (x-2) dx = 0
    static final UnivariateFunction F_SIN = Math::sin; // ∫_0^{2π} sin = 2
    public void Vtest() {
        final double REL = 1e-10;
        final double ABS = 1e-10;
        final int MIN_ITER = 1;
        final int MAX_ITER = 10;
        final int BIG_BUDGET = 1 << Math.min(28, (MAX_ITER + 12));
        RombergIntegrator R = new RombergIntegrator(REL, ABS, MIN_ITER, MAX_ITER);
        // VT01: f == null -> NullPointerException
        expectThrow("VT01 f=null",

```

```

        () -> R.integrate(1000, null, 0.0, 1.0),
        NullArgumentException.class);
// VT02: maxEval <= 0 -> NotStrictlyPositiveException
expectThrow("VT02 maxEval=0",
        () -> R.integrate(0, F_SIN, 0.0, 1.0),
        NotStrictlyPositiveException.class);
expectThrow("VT02b maxEval<0",
        () -> R.integrate(-5, F_SIN, 0.0, 1.0),
        NotStrictlyPositiveException.class);
// VT03: rel<=0 AND abs<=0 (ctor) -> NotStrictlyPositiveExcepti
on
expectThrow("VT03 rel<=0 AND abs<=0 (ctor)",
        () -> new RombergIntegrator(0.0, 0.0, 1, 10),
        NotStrictlyPositiveException.class);
// VT04: minIter < 1 (ctor) -> NumberIsTooSmallException
expectThrow("VT04 minIter<1 (ctor)",
        () -> new RombergIntegrator(1e-8, 1e-8, 0, 10),
        NumberIsTooSmallException.class);
// VT05: maxIter > 32 (ctor) -> NumberIsTooLargeException
expectThrow("VT05 maxIter>32 (ctor)",
        () -> new RombergIntegrator(1e-8, 1e-8, 1, 64),
        NumberIsTooLargeException.class);
// VT06: minIter == maxIter (ctor) -> NumberIsTooSmallException
expectThrow("VT06 minIter==maxIter (ctor)",
        () -> new RombergIntegrator(1e-8, 1e-8, 5, 5),
        NumberIsTooSmallException.class);
// VT07: NaN endpoints -> NotANumberException
expectThrow("VT07 NaN at min",
        () -> R.integrate(1000, F_LIN, Double.NaN, 1.0),
        NotANumberException.class);
// VT08: Infinite endpoints -> NotFiniteNumberException
expectThrow("VT08 -Infinity at min",
        () -> R.integrate(1000, F_SIN, Double.NEGATIVE_INFINITY
, 1.0),
        NotFiniteNumberException.class);
// VT09: min ≥ max (strict) -> NumberIsTooLargeException
expectThrow("VT09 min==max",
        () -> R.integrate(1000, F_CONST, 3.14, 3.14),
        NumberIsTooLargeException.class);
// VT10:
try {
    RombergIntegrator R_tight = new RombergIntegrator(1e-
30, 1e-30, 1, 2);

```

```

        expectThrow("VT10 hit max iterations (tight accuracy, small
band)",
                    () -> R_tight.integrate(1 << 26, F_SIN, 0.0, 100.0)
,
                    MaxCountExceededException.class);
    } catch (Throwable t) {
        System.out.println("VT10 note: " + t.getClass().getSimpleName() + " - " + t.getMessage());
    }
    // VT11
    headline("VT11 near-degenerate but valid");
    double a16 = 3.0;
    double eps16 = Math.max(1e-9, 16 * Math.ulp(a16));
    double r16 = R.integrate(BIG_BUDGET, F_CONST, a16, a16 + eps16)
;
    System.out.printf("VT11 result=%.17g (expected=%g, eps=%g)%n",
r16, 5.0 * eps16, eps16);
    // VT12
    headline("VT12 else-
path for (i >= minIter): use minIter=3 so i=1,2 take else");
    RombergIntegrator R_min3 = new RombergIntegrator(1e-8, 1e-
8, 3, 6);
    double vt23 = R_min3.integrate(1 << 20, F_CONST, 0.0, 1e-3);
    System.out.printf("VT12 result=%.17g (expected=%g)%n", vt23, 5.
0 * 1e-3);
    // VT13
    headline("VT13 tight accuracy on tiny interval");
    RombergIntegrator Rtiny = new RombergIntegrator(1e-14, 1e-
14, 1, 10);
    double r18 = Rtiny.integrate(BIG_BUDGET, F_SIN, 0.0, 1e-6);
    System.out.printf("VT13 result=%.17g%n", r18);
    // VT14
    headline("VT14 small iter band but feasible (CONST)");
    RombergIntegrator Rband = new RombergIntegrator(1e-8, 1e-
8, 1, 4);
    double r19 = Rband.integrate(1 << 18, F_CONST, -1e-4, 1e-4);
    System.out.printf("VT14 result=%.17g (expected=%g)%n", r19, 5.0
* (2e-4));
    // VT15
    headline("VT15 medium interval LIN [-5,7.25]");
    double r20 = R.integrate(Math.max(BIG_BUDGET, 1 << 21), F_LIN,
-5.0, 7.25);
    System.out.printf("VT15 result=%.17g%n", r20);
    // VT16

```

```

        try {
            headline("VT16 ctor(minIter,maxIter) boundary success (1,32)");
            RombergIntegrator R_mOnly_ok = new RombergIntegrator(1, 32);
            ;
            double vt21 = R_mOnly_ok.integrate(1 << 22, F_CONST, 0.0, 1e-4);
            System.out.printf("VT16 result=%.17g (expected≈%g)%n", vt21, 5.0 * 1e-4);
        } catch (NoSuchMethodError | IllegalArgumentException e) {
            System.out.println("VT16 note: (minIter,maxIter) ctor not present in this build.");
        } catch (Throwable t) {
            if (!(t instanceof NoSuchMethodError)) {
                System.out.println("VT16 ctor present, run result above.");
            }
        }
    }
    // VT17
    try {
        expectThrow("VT17 ctor(minIter,maxIter) maxIter>32",
            () -> {
                RombergIntegrator tmp = new RombergIntegrator(1, 33);
                if (tmp == null) System.out.println();
            },
            NumberIsTooLargeException.class);
    } catch (Throwable t) {
        if (t instanceof NoSuchMethodError) {
            System.out.println("VT17 note: (minIter,maxIter) ctor not present in this build.");
        }
    }
    System.out.println("\n[Done] RombergIntegrator V-method cases executed.");
}
//FT
class RIFT {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n=== " + name + " ===");
    }
}

```

```

    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }

    // ----- sample functions -----
    static final UnivariateFunction F_ZERO = x -> 0.0; // integral = 0
    static final UnivariateFunction F_CONST = x -> 5.0; //  $\int_0^5 5 dx = 5(b-a)$ 
    static final UnivariateFunction F_LIN = x -> x - 2.0; //  $\int_0^4 (x-2) dx = 0$ 
    static final UnivariateFunction F_SIN = Math::sin; //  $\int_0^{2\pi} \sin = 2$ 

    public void Ftest() {
        final double REL = 1e-10;
        final double ABS = 1e-10;
        final int MIN_ITER = 1;
        final int MAX_ITER = 10;
        final int BIG_BUDGET = 1 << Math.min(28, (MAX_ITER + 12));
        RombergIntegrator R = new RombergIntegrator(REL, ABS, MIN_ITER, MAX_ITER);
        // FT01: reversed interval with huge magnitudes -> NumberIsTooLargeException
        expectThrow("FT01 reversed extreme interval",
            () -> R.integrate(1000, F_SIN, 5.005341669616643E307, -1.1209145429141500E308),
            NumberIsTooLargeException.class);
        // FT02: degenerate interval at extreme negative -> NumberIsTooLargeException
        expectThrow("FT02 degenerate interval (min==max, extreme)",
            () -> R.integrate(1000, F_CONST,

```

```

        -1.7976931348623157E308, -
1.7976931348623157E308),
        NumberIsTooLargeException.class);
// FT03: NaN at min -> NotANumberException
expectThrow("FT03 NaN min",
        () -> R.integrate(1000, F_LIN, Double.NaN, 1.0),
        NotANumberException.class);
// FT04: NaN at max -> NotANumberException
expectThrow("FT04 NaN max",
        () -> R.integrate(1000, F_SIN, 0.0, Double.NaN),
        NotANumberException.class);
// FT05: +Infinity at max -> NotFiniteNumberException
expectThrow("FT05 +Infinity at max",
        () -> R.integrate(1000, F_SIN, -
10.0, Double.POSITIVE_INFINITY),
        NotFiniteNumberException.class);
// FT06: -Infinity at min -> NotFiniteNumberException
expectThrow("FT06 -Infinity at min",
        () -> R.integrate(1000, F_CONST, Double.NEGATIVE_INFINITY, 10.0),
        NotFiniteNumberException.class);
// FT07: tiny budget on very wide interval -> TooManyEvaluation
sException
expectThrow("FT07 budget=1 on wide interval",
        () -> R.integrate(1, F_SIN, -1.0e3, 1.0e3),
        TooManyEvaluationsException.class);
// FT08: budget=1 on [0,  $\pi$ ] -> TooManyEvaluationsException
expectThrow("FT08 budget=1 on [0, $\pi$ ]",
        () -> R.integrate(1, F_SIN, 0.0, Math.PI),
        TooManyEvaluationsException.class);
// FT09: ZERO on tiny symmetric interval (success; integral=0)
headline("FT09 ZERO tiny symmetric interval");
double ft09 = R.integrate(BIG_BUDGET, F_ZERO, -1e-12, 1e-12);
System.out.printf("FT09 result=%.17g\n", ft09);
// FT10: CONST near-degenerate at huge offset (success)
headline("FT10 CONST huge-offset small window");
double A10 = 1.2345678901234567E307;
double ft10 = R.integrate(Math.max(BIG_BUDGET, 1 << 22), F_CONST, A10, A10 + 1.0);
System.out.printf("FT10 result=%.17g (expected=%g)\n", ft10, 5.0);

// FT11: SIN on random moderate interval (success; boosted budget for Romberg)
headline("FT11 SIN moderate random window");

```

```

        int bud11 = Math.max(BIG_BUDGET, 1 << Math.min(28, (MAX_ITER +
12)));
        double ft11 = R.integrate(bud11, F_SIN, 1.23456789012345, 23.45
67890123456);
        System.out.printf("FT11 result=%.17g (budget=%d)%n", ft11, bud1
1);
        // FT12: LIN on wide finite random range (success; boosted budg
et)
        headline("FT12 LIN wide finite random");
        int bud12 = Math.max(BIG_BUDGET, 1 << Math.min(28, (MAX_ITER +
12)));
        double ft12 = R.integrate(bud12, F_LIN, -
5.43210987654321E2, 7.65432109876543E2);
        System.out.printf("FT12 result=%.17g (budget=%d)%n", ft12, bud1
2);
        // FT13: tight accuracy + very small iter band -> Likely MaxCou
ntExceededException
        try {
            RombergIntegrator R_tight_FZ = new RombergIntegrator(1e-
30, 1e-30, 1, 2);
            expectThrow("FT13 hit max iterations (tight acc, small band
)",
                () -> R_tight_FZ.integrate(Math.max(BIG_BUDGET, 1 <
< 24), F_SIN, 0.0, 100.0),
                MaxCountExceededException.class);
        } catch (Throwable t) {
            System.out.println("FT13 note: " + t.getClass().getSimpleNa
me() + " - " + t.getMessage());
        }
        // FT14: CONST random mid-size window (success)
        headline("FT14 CONST random mid-size window");
        double a14 = -7.25;
        double b14 = 9.875;
        double ft14 = R.integrate(Math.max(BIG_BUDGET, 1 << 21), F_CONS
T, a14, b14);
        System.out.printf("FT14 result=%.17g (expected≈%g)%n", ft14, 5.
0 * (b14 - a14));
        // FT15: SIN oscillatory large interval (success)
        headline("FT15 SIN oscillatory large interval");
        double ft15 = R.integrate(Math.max(BIG_BUDGET, 1 << 23), F_SIN,
-100.0, 100.0);
        System.out.printf("FT15 result=%.17g%n", ft15);
        // FT16: rel-only accuracy (abs=0) success case
        headline("FT16 rel-only accuracy (rel>0, abs=0)");

```

```

        RombergIntegrator R_relOnly = new RombergIntegrator(1e-
6, 0.0, 1, 10);
        double ft16 = R_relOnly.integrate(Math.max(BIG_BUDGET, 1 << 21)
, F_CONST, -1.0, 2.0);
        System.out.printf("FT16 result=%.17g (expected≈%g)%n", ft16, 15
.0);

        // FT17: abs-only accuracy (rel=0) success case
        headline("FT17 abs-only accuracy (rel=0, abs>0)");
        RombergIntegrator R_absOnly = new RombergIntegrator(0.0, 1e-
6, 1, 10);
        double ft17 = R_absOnly.integrate(Math.max(BIG_BUDGET, 1 << 21)
, F_LIN, -3.0, 7.0);
        System.out.printf("FT17 result=%.17g%n", ft17);
        System.out.println("\n[Done] RombergIntegrator FUZZING cases ex
ecuted.");
    }
}
//Z3
class RIZ3{
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n=== " + name + " ===");
    }
    private static void expectThrow(String name, Runnable r, Class<?> e
xpected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s%n", name, expe
cted.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " -
" + t.getMessage());
            System.out.printf("[%s] %s threw expected%n",
                t.getClass().equals(expected) ? "PASS" : "FAIL", na
me);
        }
    }
    // ----- simple functions -----
    static final UnivariateFunction F_ZERO = x -> 0.0;
    static final UnivariateFunction F_CONST = x -> 5.0;
    static final UnivariateFunction F_SIN = Math::sin;
    static final UnivariateFunction F_LIN = x -> x - 2.0;
    public void Z3test() {

```

```

    final RombergIntegrator R_OK = new RombergIntegrator(1e-3, 1e-
3, 1, 2);
    // Z301 -----
    -----
    expectThrow("Z301 f=null",
        () -> R_OK.integrate(10, null, 0.0, 1.0),
        NullArgumentException.class);
    // Z302 -----
    -----
    RombergIntegrator R302 = new RombergIntegrator(0.0, 0.5, 1, 2);
    expectThrow("Z302 maxEval<=0",
        () -> R302.integrate(0, F_ZERO, 0.0, 1.0),
        NotStrictlyPositiveException.class);
    // Z303 -----
    -----
    expectThrow("Z303 rel<=0 AND abs<=0 (ctor)",
        () -> new RombergIntegrator(0.0, 0.0, 1, 2),
        NotStrictlyPositiveException.class);
    // Z304 -----
    -----
    expectThrow("Z304 minIter<1 (ctor)",
        () -> new RombergIntegrator(0.5, 0.0, 0, 1),
        NumberIsTooSmallException.class);
    // Z305 -----
    -----
    expectThrow("Z305 maxIter>32 (ctor)",
        () -> new RombergIntegrator(0.5, 0.0, 1, 65),
        NumberIsTooLargeException.class);
    // Z306 -----
    -----
    expectThrow("Z306 minIter>=maxIter (ctor)",
        () -> new RombergIntegrator(0.0, 0.5, 1, 0),
        NumberIsTooSmallException.class);
    // Z307 -----
    -----
    RombergIntegrator R307 = new RombergIntegrator(0.5, 0.0, 1, 2);
    expectThrow("Z307 NaN at min",
        () -> R307.integrate(10, F_LIN, Double.NaN, 1.0),
        NotANumberException.class);
    // Z308 -----
    -----
    RombergIntegrator R308 = new RombergIntegrator(0.0, 0.5, 1, 2);
    expectThrow("Z308 NaN at max",
        () -> R308.integrate(10, F_LIN, 0.0, Double.NaN),

```

```

        NotANumberException.class);
// Z309 -----
-----
RombergIntegrator R309 = new RombergIntegrator(0.0, 0.5, 1, 2);
expectThrow("Z309 -Infinity(min)",
    () -> R309.integrate(10, F_SIN, Double.NEGATIVE_INFINITY,
Y, 1.0),
        NotFiniteNumberException.class);
// Z310 -----
-----
RombergIntegrator R310 = new RombergIntegrator(0.0, 0.5, 1, 2);
expectThrow("Z310 +Infinity(max)",
    () -> R310.integrate(10, F_SIN, 0.0, Double.POSITIVE_INFINITY),
        NotFiniteNumberException.class);
// Z311 -----
-----
RombergIntegrator R311 = new RombergIntegrator(0.5, 0.0, 1, 2);
expectThrow("Z311 min==max",
    () -> R311.integrate(10, F_CONST, 0.0, 0.0),
        NumberIsTooLargeException.class);
// Z312 -----
-----
RombergIntegrator R312 = new RombergIntegrator(0.5, 0.0, 1, 2);
expectThrow("Z312 budget exhausted (maxEval=1 surrogate)",
    () -> R312.integrate(1, F_SIN, 0.0, 1.0),
        TooManyEvaluationsException.class);
// Z313 -----
-----
try {
    RombergIntegrator R313 = new RombergIntegrator(1e-30, 1e-
30, 1, 2);
    expectThrow("Z313 hit max iterations (tight acc, small band
)",
        () -> R313.integrate(1 << 24, F_SIN, 0.0, 100.0),
            MaxCountExceededException.class);
} catch (Throwable t) {
    System.out.println("Z313 note: " + t.getClass().getSimpleName() + " - " + t.getMessage());
}
// Z314 -----
-----
headline("Z314 success (value accuracy surrogate)");
RombergIntegrator R314 = new RombergIntegrator(0.5, 0.0, 1, 4);

```

```

        double z314 = R314.integrate(1 << 20, F_CONST, 0.0, 1e-4);
        System.out.printf("Z314 result=%.17g (expected=%g)%n", z314, 5.
0 * 1e-4);
        // Z315 -----
        headline("Z315 success (interval shrinkage surrogate)");
        RombergIntegrator R315 = new RombergIntegrator(0.0, 0.5, 1, 4);
        double z315 = R315.integrate(1 << 20, F_ZERO, -1e-6, 1e-6);
        System.out.printf("Z315 result=%.17g%n", z315);
        // Z316 -----
        headline("Z316 success (both flags surrogate; within budget)");
        RombergIntegrator R316 = new RombergIntegrator(0.0, 0.5, 1, 4);
        double z316 = R316.integrate(10, F_ZERO, -1.0, 1.0);
        System.out.printf("Z316 result=%.17g%n", z316);
        // Z317 -----
        headline("Z317 success (tight interval ≤1e-6)");
        RombergIntegrator R317 = new RombergIntegrator(0.5, 0.0, 1, 4);
        double z317 = R317.integrate(1000, F_CONST, 3.0, 3.0 + 1e-6);
        System.out.printf("Z317 result=%.17g (expected=%g)%n", z317, 5.
0 * 1e-6);
        System.out.println("\n[Done] Z301-Z317 executed.");
    }
}

class RITest {
    public static void main(String[] args) {
        // VT
        RIVT t1 = new RIVT();
        t1.Vtest();
        // FT
        RIFT t2 = new RIFT();
        // t2.Ftest();
        // Z3
        RIZ3 t3 = new RIZ3();
        // t3.Z3test();
    }
}

```

- Coverage report

1. Test case number: 17
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ RombergIntegrator.java	58.3 %	14	10	24
> RIFT	0.0 %	0	2	2
> RIZ3	0.0 %	0	2	2
> RIVT	25.0 %	2	6	8
▼ RombergIntegrator	100.0 %	12	0	12
RombergIntegrator(double, double,	100.0 %	2	0	2
RombergIntegrator(int, int)	100.0 %	2	0	2
doIntegrate()	100.0 %	8	0	8
> Rltest		0	0	0

3. V-Method testing instruction coverage: 97%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ RombergIntegrator.java	35.2 %	587	1,080	1,667
> RIFT	2.4 %	12	490	502
> RIZ3	2.5 %	12	460	472
> RIVT	76.1 %	388	122	510
> Rltest	83.3 %	15	3	18
▼ RombergIntegrator	97.0 %	160	5	165
RombergIntegrator(double, double,	100.0 %	19	0	19
RombergIntegrator(int, int)	100.0 %	17	0	17
doIntegrate()	100.0 %	124	0	124

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ RombergIntegrator.java	37.5 %	9	15	24
> RIVT	0.0 %	0	8	8
▼ RombergIntegrator	58.3 %	7	5	12
RombergIntegrator(double, double,	50.0 %	1	1	2
doIntegrate()	75.0 %	6	2	8
> RIFT	100.0 %	2	0	2
> Rltest		0	0	0

5. Fuzz testing instruction coverage: 81.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ RombergIntegrator.java	19.9 %	332	1,335	1,667
> RIVT	2.4 %	12	498	510
> RIFT	35.5 %	178	324	502
> Rltest	44.4 %	8	10	18
▼ RombergIntegrator	81.2 %	134	31	165
RombergIntegrator(double, double,	52.6 %	10	9	19
doIntegrate()	100.0 %	124	0	124

6. Z3 branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ RombergIntegrator.java	33.3 %	8	16	24
> RIFT	0.0 %	0	2	2
> RIVT	0.0 %	0	8	8
▼ RombergIntegrator	50.0 %	6	6	12
doIntegrate()	50.0 %	4	4	8
RombergIntegrator(double, double,	100.0 %	2	0	2
> RIZ3	100.0 %	2	0	2
> Rltest		0	0	0

7. Z3 instruction coverage: 81.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ RombergIntegrator.java	32.5 %	542	1,125	1,667
> RIVT	2.4 %	12	498	510
> RIFT	2.4 %	12	490	502
> RIZ3	78.0 %	368	104	472
▼ RombergIntegrator	81.8 %	135	30	165
doIntegrate()	93.5 %	116	8	124
RombergIntegrator(double, double,	100.0 %	19	0	19
> Rltest	83.3 %	15	3	18